UNIVERSIDAD PRIVADA FRANZ TAMAYO

CARRERA INGENIERÍA DE SISTEMAS

Examen Práctico Hito 3

Caja negra · Caja blanca · Gherkin · Bugs · GitHub

Aplicación bajo prueba: SauceDemo / Swag Labs (www.saucedemo.com)

AUTOR:

Marcos Alexander Panozo Patty

DOCENTE:

Reynel Sánchez Chuquimia

FECHA:

20 – 10 - 2026

TESTING, QUALITY ASSURANCE (TQA -711)

LA PAZ - BOLIVIA

2026

1. Introducción

El presente informe documenta el Examen Práctico del Hito 3 de Testing & Quality Assurance. Se aplicaron técnicas de caja negra sobre la aplicación de demostración **SauceDemo (Swag Labs)** — https://www.saucedemo.com/ —, se analizó un fragmento de código Python con técnicas de caja blanca, se redactaron escenarios BDD en Gherkin y se documentaron los defectos encontrados. Todo el trabajo se encuentra versionado en el repositorio de GitHub **Examen-Hito-3-TQA**.

Las pruebas se ejecutaron el 20-10-2026 en un navegador Chromium de escritorio, con los usuarios de demostración `standard_user`, `locked_out_user` y `problem_user` (contraseña `secret_sauce`).

2. Casos de prueba de Caja Negra — SauceDemo (Swag Labs)

URL probada: https://www.saucedemo.com/
Fecha de ejecución: 20-10-2026
Navegador: Chromium (escritorio)
Contraseña común de los usuarios demo: `secret_sauce`

ID	Título	Funcionalidad	Estado
CN-01	Inicio de sesión con credenciales válidas	Login	✅ Passed
CN-02	Inicio de sesión con usuario bloqueado	Login	✅ Passed
CN-03	Agregar y eliminar productos del carrito	Carrito	✅ Passed
CN-04	Checkout completo con validación de campo obligatorio	Checkout	✅ Passed
CN-05	Checkout con `problem_user` — ingreso de datos del comprador	Checkout	❌ Failed (BUG-01)

CN-01 — Inicio de sesión con credenciales válidas

- **Objetivo:** Verificar que un usuario válido accede al catálogo de productos.
- **Precondiciones:** Estar en la página de login, sin sesión iniciada.
- **Datos de prueba:** Usuario `standard_user` / Contraseña `secret_sauce`.
- **Pasos:**
    1. Abrir https://www.saucedemo.com/
    2. Ingresar `standard_user` en *Username*.
    3. Ingresar `secret_sauce` en *Password*.
    4. Presionar **Login**.

- **Resultado esperado:** Redirección a `/inventory.html`, título “Products” y listado de productos.
- **Resultado obtenido:** Se redirigió a `/inventory.html`, se mostró el título “Products” con 6 productos.
- **Estado:** ✅ Passed

CN-02 — Inicio de sesión con usuario bloqueado

- **Objetivo:** Verificar que un usuario bloqueado no puede ingresar y recibe un mensaje claro.
- **Precondiciones:** Estar en la página de login, sin sesión iniciada.
- **Datos de prueba:** Usuario `locked_out_user` / Contraseña `secret_sauce`.
- **Pasos:**
    1. Abrir https://www.saucedemo.com/
    2. Ingresar `locked_out_user` y `secret_sauce`.
    3. Presionar **Login**.
- **Resultado esperado:** El sistema permanece en el login y muestra un mensaje de error indicando que el usuario está bloqueado.
- **Resultado obtenido:** Permaneció en `/` y mostró: *“Epic sadface: Sorry, this user has been locked out.”*
- **Estado:** ✅ Passed

CN-03 — Agregar y eliminar productos del carrito

- **Objetivo:** Verificar que el carrito refleja correctamente productos agregados y eliminados.
- **Precondiciones:** Sesión iniciada con `standard_user`, carrito vacío.
- **Datos de prueba:** Productos *Sauce Labs Backpack* ($29.99) y *Sauce Labs Bike Light* ($9.99).
- **Pasos:**
    1. En el catálogo, presionar **Add to cart** en *Sauce Labs Backpack*.
    2. Presionar **Add to cart** en *Sauce Labs Bike Light*.
    3. Verificar el contador del ícono del carrito.
    4. Abrir el carrito.
    5. Presionar **Remove** en *Sauce Labs Bike Light*.
- **Resultado esperado:** El contador muestra 2; el carrito lista ambos productos con su precio; tras eliminar, queda solo *Backpack* y el contador baja a 1.
- **Resultado obtenido:** Contador = 2; carrito con *Backpack $29.99* y *Bike Light $9.99*; tras **Remove**, solo *Backpack* y contador = 1.
- **Estado:** ✅ Passed

CN-04 — Checkout completo con validación de campo obligatorio

- **Objetivo:** Verificar que el checkout valida el nombre obligatorio y que una compra válida se completa con totales correctos.
- **Precondiciones:** Sesión con `standard_user`, *Sauce Labs Backpack* en el carrito.
- **Datos de prueba:** First Name: *(vacío)* y luego `Marcos`; Last Name: `Perez`; Zip: `0000`.
- **Pasos:**

1. Abrir el carrito y presionar **Checkout**.
2. Dejar *First Name* vacío, llenar *Last Name* y *Zip* y presionar **Continue**.
3. Llenar *First Name* con `Marcos` y presionar **Continue**.
4. Verificar el resumen (Item total, Tax, Total).
5. Presionar **Finish**.

- **Resultado esperado:** Paso 2: mensaje "Error: First Name is required". Paso 4: Item total $29.99, impuesto y total coherentes. Paso 5: mensaje de confirmación de orden.
- **Resultado obtenido:** Paso 2: *"Error: First Name is required"*. Paso 4: Item total $29.99, Tax $2.40, Total $32.39 (29.99 + 2.40 = 32.39 ✔). Paso 5: *"Thank you for your order!"* en `/checkout-complete.html`.
- **Estado:** ✅ Passed

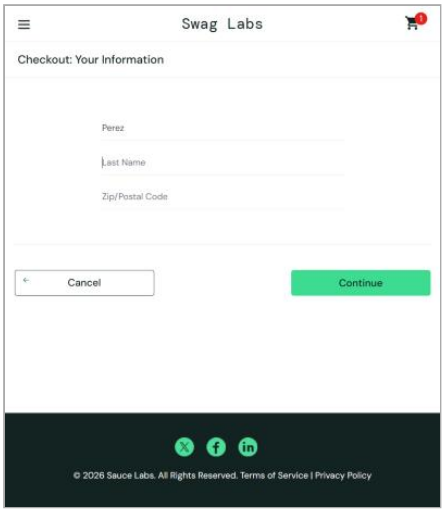
CN-05 — Checkout con `problem_user`: ingreso de datos del comprador

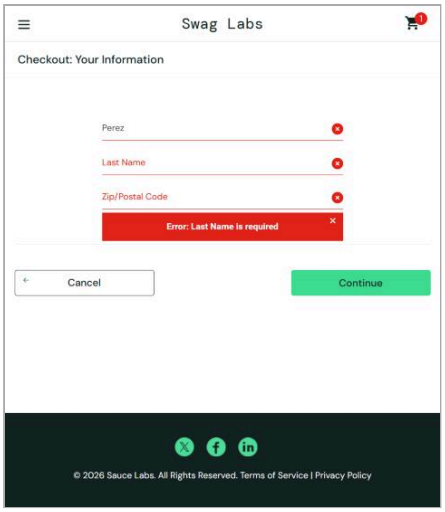
- **Objetivo:** Verificar que el formulario de checkout conserva los datos ingresados en cada campo.
- **Precondiciones:** Sesión con `problem_user`, *Sauce Labs Backpack* en el carrito.
- **Datos de prueba:** First Name `Marcos`, Last Name `Perez`.
- **Pasos:**
  1. Abrir el carrito y presionar **Checkout**.
  2. Hacer clic en *First Name* y escribir `Marcos`.
  3. Hacer clic en *Last Name* y escribir `Perez`.
  4. Presionar **Continue**.
- **Resultado esperado:** First Name = "Marcos", Last Name = "Perez"; al continuar solo debería pedir el Zip.
- **Resultado obtenido:** El texto escrito en *Last Name* aparece en *First Name* ("Perez") y *Last Name* queda vacío. Al continuar: *"Error: Last Name is required"*. No es posible completar la compra.
- **Estado:** ❌ Failed → ver **BUG-01**
- **Evidencia:** `evidencias/capturas/BUG-01_lastname_sobrescribe_firstname.jpg`, `evidencias/capturas/BUG-01_error_last_name_required.jpg`

3. Bugs encontrados — SauceDemo

BUG-01 — Con `problem_user`, el campo “Last Name” sobrescribe “First Name” en el checkout

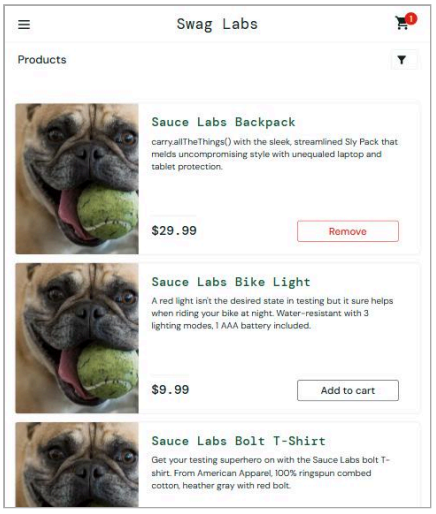
- **Caso relacionado:** CN-05
- **Severidad:** Alta (bloquea completar la compra) · **Prioridad:** Alta
- **Descripción:** En *Checkout: Your Information*, al escribir en *Last Name* el texto se coloca en *First Name* y *Last Name* permanece vacío, por lo que la validación impide continuar.
- **Precondiciones / datos:** Sesión con `problem_user` / `secret_sauce`; un producto en el carrito. First Name `Marcos`, Last Name `Perez`.
- **Pasos para reproducir:**
  1. Login con `problem_user`.
  2. Agregar *Sauce Labs Backpack* al carrito, abrir carrito y presionar **Checkout**.
  3. Escribir `Marcos` en *First Name*.
  4. Escribir `Perez` en *Last Name*.
  5. Presionar **Continue**.
- **Resultado esperado:** First Name = “Marcos”, Last Name = “Perez”; avanzar o solicitar solo el Zip.
- **Resultado actual:** First Name = “Perez”, Last Name vacío; mensaje *“Error: Last Name is required”*. Reproducible en cada intento.
- **Evidencia:**





BUG-02 — Con `problem_user`, todos los productos muestran la misma imagen incorrecta

- **Caso relacionado:** observado en la precondición de CN-05 (catálogo)
- **Severidad:** Media · **Prioridad:** Media
- **Descripción:** En `/inventory.html` los 6 productos muestran la misma imagen (un perro), que no corresponde a ningún producto. El recurso cargado es `/assets/sl-404-…jpg` para todos.
- **Precondiciones / datos:** Sesión con `problem_user` / `secret_sauce`.
- **Pasos para reproducir:**
    1. Login con `problem_user`.
    2. Observar el catálogo de productos.
- **Resultado esperado:** Cada producto muestra su propia imagen (mochila, luz de bicicleta, polera, etc.).
- **Resultado actual:** Las 6 tarjetas muestran la misma imagen genérica `sl-404`.
- **Evidencia:**



4. Análisis de Caja Blanca — `validar_compra`

```
def validar_compra(edad, cuenta_activa, saldo, monto):
    if edad < 18:                          # D1
        return "Compra no permitida: edad"     # S1
    if not cuenta_activa:                  # D2
        return "Compra no permitida: cuenta"   # S2
    if saldo >= monto and monto > 0:       # D3
        return "Compra aprobada"               # S3
    return "Compra rechazada"                  # S4
```

1. Decisiones y condiciones simples

Decisión	Expresión	Condiciones simples
D1	`edad < 18`	C1: `edad < 18`
D2	`not cuenta_activa`	C2: `cuenta_activa` (negada)
D3	`saldo >= monto and monto > 0`	C3: `saldo >= monto`, C4: `monto > 0`

Total: 3 decisiones y 4 condiciones simples. Hay 4 sentencias `return` (S1–S4), es decir 4 caminos posibles.

2. Criterio utilizado: **Cobertura de decisiones**

Cada decisión debe evaluarse al menos una vez como **Verdadera** y una vez como **Falsa**.

3. Mínimo de casos y cálculo

Las decisiones están en secuencia con `return` temprano: si D1 es V el programa termina, así que D2 y D3 solo se evalúan cuando las anteriores son F.

- D1 = V → 1 caso (sale por S1).
- D1 = F y D2 = V → 1 caso (S2).
- D1 = F, D2 = F, D3 = V → 1 caso (S3).
- D1 = F, D2 = F, D3 = F → 1 caso (S4).

Ningún caso puede cubrir dos resultados “V” a la vez, porque cada V termina la función. Por eso el **mínimo = 4 casos** (número de decisiones + 1 = 3 + 1, que coincide con la complejidad ciclomática V(G) = 4). Este mismo conjunto también logra la cobertura de sentencias (S1–S4).

4. Tres casos diseñados

ID	edad	cuenta_activa	saldo	monto	Resultado esperado	Cubre
CB-01	16	True	100	50	“Compra no permitida: edad”	D1 = V (S1)
CB-02	25	False	100	50	“Compra no permitida: cuenta”	D1 = F, D2 = V (S2)
CB-03	25	True	100	50	“Compra aprobada”	D1 = F, D2 = F, D3 = V (S3)

5. ¿Alcanzan el criterio? Ajuste necesario

No. Con 3 casos D3 nunca toma el valor **Falso** y la sentencia S4 (`"Compra rechazada"`) no se ejecuta. Se cubren 5 de 6 resultados de decisión (83 %).

Ajuste: agregar un cuarto caso:

ID	edad	cuenta_activa	saldo	monto	Resultado esperado	Cubre
CB-04	25	True	30	50	“Compra rechazada”	D3 = F (S4)

Con CB-01…CB-04 se logra 100 % de cobertura de decisiones y de sentencias.

6. Nota: diferencia con cobertura de condiciones

La cobertura de condiciones exige que **cada condición simple** (C1–C4) sea V y F. En D3, CB-03 hace C3 = V y C4 = V, y CB-04 hace C3 = F; pero como Python evalúa `and` en cortocircuito, C4 no se evalúa cuando C3 es F. Para que C4 sea Falsa haría falta un caso adicional:

ID	edad	cuenta_activa	saldo	monto	Resultado esperado	Cubre
CB-05	25	True	100	0	“Compra rechazada”	C3 = V, C4 = F

Por eso, para cobertura de condiciones el mínimo sube a **5 casos**, lo que muestra que cobertura de decisiones ≠ cobertura de condiciones.

5. Escenarios Gherkin (BDD) — SauceDemo

```
Feature: Inicio de sesión en Swag Labs
  Como cliente de la tienda
  Quiero iniciar sesión con mi usuario
  Para poder ver y comprar productos

  Background:
    Given que estoy en la página de login "https://www.saucedemo.com/"

  Scenario: Acceso exitoso con credenciales válidas
    When ingreso el usuario "standard_user"
    And ingreso la contraseña "secret_sauce"
    And presiono el botón "Login"
    Then debería ser redirigido a la página "/inventory.html"
    And debería ver el título "Products"

  Scenario: Acceso denegado a usuario bloqueado
    When ingreso el usuario "locked_out_user"
    And ingreso la contraseña "secret_sauce"
    And presiono el botón "Login"
    Then debería permanecer en la página de login
    And debería ver el mensaje "Epic sadface: Sorry, this user has been locked
out."
```

```
Feature: Checkout de compra
  Como cliente con productos en el carrito
  Quiero completar mis datos de envío
  Para finalizar mi compra

  Background:
    Given que inicié sesión como "standard_user"
    And tengo el producto "Sauce Labs Backpack" en el carrito
    And estoy en la página "Checkout: Your Information"

  Scenario: Compra completada con datos válidos
    When ingreso "Marcos" en First Name
    And ingreso "Perez" en Last Name
    And ingreso "0000" en Zip/Postal Code
    And presiono "Continue"
    Then debería ver el total "Total: $32.39"
    When presiono "Finish"
    Then debería ver el mensaje "Thank you for your order!"

  Scenario: Nombre obligatorio vacío
    When dejo vacío el campo First Name
    And ingreso "Perez" en Last Name
    And ingreso "0000" en Zip/Postal Code
    And presiono "Continue"
    Then debería ver el mensaje "Error: First Name is required"
    But no debería avanzar al resumen de la compra
```

Gherkin describe el comportamiento esperado; por sí solo no automatiza las pruebas (se necesitaría una herramienta como Behave o Cucumber).

6. Repositorio GitHub y organización del trabajo

Repositorio: https://github.com/AlexPanozo27/Examen-Hito-3-TQA

El trabajo se versionó con Git desde la terminal (git init, add, commit, remote y push) y se publicó en un repositorio individual. El docente fue agregado como colaborador mediante el correo institucional doc.reynel.sanchez.ch@unifranz.edu.bo.

Estructura del repositorio

```
Examen-Hito-3-TQA/
├── README.md
├── docs/
│   ├── casos_caja_negra.md
│   ├── analisis_caja_blanca.md
│   └── escenarios_gherkin.md
├── bugs/
│   └── bugs_encontrados.md
├── evidencias/
│   └── capturas/
└── entregable/
    └── informe_hito3.pdf
```

Historial de commits

#	Mensaje del commit
1	Inicializa repositorio con README del Examen Hito 3
2	Agrega 5 casos de prueba de caja negra ejecutados en SauceDemo
3	Documenta BUG-01 y BUG-02 con evidencias (problem_user)
4	Agrega análisis de caja blanca de validar_compra (cobertura de decisiones)
5	Agrega escenarios Gherkin de login y checkout
6	Agrega informe final consolidado en PDF

Issues y Project Board

Se creó el tablero **“Examen Hito 3 TQA - Board”** (GitHub Projects, vista Board con columnas Todo / In Progress / Done) y se registraron Issues para cada tarea y para cada defecto:

Issue	Tipo
Diseñar 5 casos de caja negra · Ejecutar pruebas de caja negra · Revisar evidencias	Caja negra
Analizar código de caja blanca · Registrar resultados de caja blanca	Caja blanca
Redactar escenarios Gherkin	Gherkin
Documentar bugs · [BUG-01] · [BUG-02]	Defectos
Actualizar README · Preparar informe PDF · Revisar informe final	Documentación

7. Conclusiones

- Con `standard_user` las funciones principales (login, carrito, checkout y validación de campos) funcionan correctamente: 4 de 5 casos de caja negra pasaron.
- Con `problem_user` se detectaron 2 defectos reproducibles; el BUG-01 es de severidad alta porque impide completar la compra.
- En caja blanca, 3 casos no bastan para la cobertura de decisiones: el mínimo es 4 (D3 debe ser falsa). Para cobertura de condiciones el mínimo sube a 5 por la evaluación en cortocircuito del `and`.
- Gherkin permitió expresar el comportamiento esperado en lenguaje claro y verificable, útil como base para automatizar en el futuro.
- Git y GitHub (commits, Issues y Project Board) permitieron organizar y dar trazabilidad a todo el trabajo.